

RISC-V S-level Physical Memory Protection for Hypervisor

Editor - José Martins and Bicheng Yang

Version 0.6, 09/2026: This document is in development. Assume everything can change. See http://riscv.org/spec-state for details.

Table of Contents

Preamble **1**
Copyright and license information **2**
Contributors **3**
1. Introduction **4**
2. "Ssvspmp" Extension for VS-level Physical Memory Protection (vSPMP) **5**
2.1. Extension Dependencies 5
2.2. Virtual SPMP (vSPMP) 5
2.3. Access Method for vSPMP registers 6
3. "Ssvspmpen" Extension for Optimizing Context Switching of vSPMP Entries **8**
4. "Sshspmpdeleg" Extension for Sharing Hardware Resources between SPMP and vSPMP **9**
4.1. Resource Sharing between SPMP and vSPMP 9
5. "Smhspmpdeleg" Extension for M-mode Support of vSPMP Resource Sharing **11**
5.1. Extension to `mpmpdeleg` 11
5.2. M-mode Support for Hypervisor/Guest SPMP Resource Sharing 12
6. "Sshspmpen" Extension for Optimizing Context Switching of SPMP Hypervisor/Guest Entries **15**

Preamble



This document is in the Development state

Assume everything can change. This draft specification will change before being accepted as standard, so implementations made to this draft specification will likely not conform to the future standard.

Copyright and license information

This specification is licensed under the Creative Commons Attribution 4.0 International License (CC-BY 4.0). The full license text is available at creativecommons.org/licenses/by/4.0/.

Copyright 2026 by RISC-V International.

Contributors

The following individuals have contributed, directly or indirectly, to the proposed SPMP for Hypervisor specifications:

(Contributors are sorted alphabetically by last name.)

- José Martins, Editor <jose@osyx.tech>
- Bicheng Yang, Editor <bichengyang@sjtu.edu.cn>
- Haibo Chen
- Luís Cunha
- Ruud Derwig
- Dong Du
- Lupe Carlos IV
- Radim Krčmář
- Kajetan Nürnberger
- Sandro Pinto
- Manuel Rodríguez
- Thomas Roecker
- Bruno Sá
- Peter Vrabel
- Harry Wagstaff
- Yubin Xia

Chapter 1. Introduction

The RISC-V S-level Physical Memory Protection (SPMP) extension enables an S-mode operating system to isolate its U-mode tasks in environments where supervisor mode is implemented but virtual memory is not, as is common in the IoT and automotive domains, where an MMU is frequently avoided to comply with stringent constraints on system resources and execution latency. The same domains increasingly rely on virtualization to consolidate formerly distributed microcontrollers into fewer, more capable devices, such as the zone controllers of modern automotive E/E architectures, time-multiplexing real-time operating systems and mixed-criticality workloads on a single hart while preserving their isolation and worst-case execution time guarantees. Real-time hypervisors rely on two-stage memory protection for this purpose. One stage isolates virtual machines from each other, the other isolates the mixed-criticality tasks within each guest.

A guest operating system running in VS-mode has the same need for SPMP as when running natively, to isolate its VU-mode tasks. Without hardware support, the hypervisor must trap and emulate every guest access to the SPMP state, folding the guest's rules into the single SPMP alongside its own. Since the SPMP does not distinguish VS-mode from VU-mode accesses, treating both as U-mode accesses, the hypervisor would furthermore have to intercept every transition between VS-mode and VU-mode and reprogram the SPMP accordingly. This adds complexity to the hypervisor, and it adds latency and jitter to the guest's task context switches, at odds with the real-time guarantees the guest, often an RTOS, is expected to provide.

The RISC-V S-level Physical Memory Protection for Hypervisor extensions address this by integrating the SPMP and Hypervisor extensions into a two-stage SPMP. The first stage is the virtual SPMP (vSPMP), under the control of VS-mode, which allows a guest supervisor to specify memory protection policies within its guest physical address space without hypervisor intervention. Guest VS/VU-mode accesses are further checked by the SPMP, as defined by the RISC-V S-level Physical Memory Protection (SPMP) specification, Version 1.0. The extensions replicate for VS-mode the SPMP support available to HS-mode, with as few changes as possible to the structure and functionality of the base extensions, and complement it with enhancements for the hypervisor, such as the delegation of hardware entries between the SPMP and the vSPMP and the separate control of the SPMP entries applied to guest accesses.

Chapter 2. "Ssvspmp" Extension for VS-level Physical Memory Protection (vSPMP)

The `Ssvspmp` extension introduces a new memory protection layer called Virtual SPMP (vSPMP), a replica of the SPMP placed under the control of VS-mode so that a guest can enforce memory protection within its own guest physical address space. Together with the SPMP, which keeps checking guest accesses under the control of HS-mode, it establishes a two-stage SPMP hierarchy whose checks logically precede those of PMP or ePMP, as shown in Figure 1. Where page-based virtual memory is implemented, either stage can be replaced by address translation when the corresponding translation mode is not `Bare`, as described in Table 1.

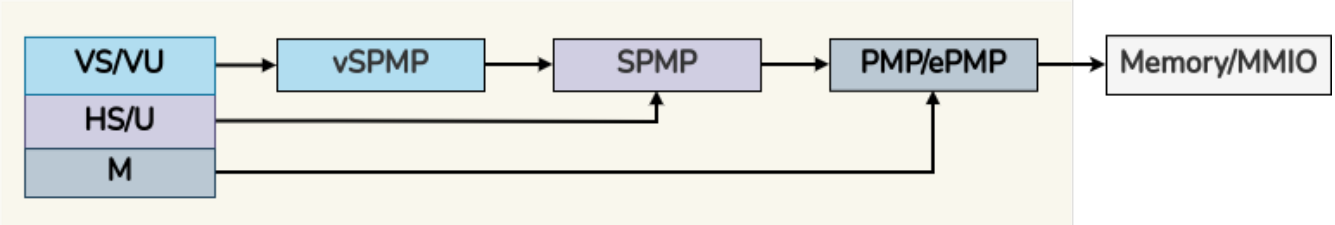


Figure 1. Dual-Stage SPMP: logical path when vsatp.MODE and hgatp.MODE are Bare

2.1. Extension Dependencies

1. The `Sspmp` extension, Version 1.0, must be implemented.
2. The `Shbare` extension must be implemented.
3. The `vsstatus.SUM` (permit Supervisor User Memory access) bit must be **writable**.
4. The `vsstatus.MXR` (Make eXecutable Readable) bit must be **writable**.

2.2. Virtual SPMP (vSPMP)

The vSPMP behaves as the SPMP defined by `Sspmp`, with the following substitutions:

1. It is under the control of VS-mode.
2. It enforces protection only for memory accesses whose effective privilege mode is VS or VU.
3. VS-mode accesses are subject to the rules `Sspmp` defines for S-mode accesses, and VU-mode accesses to those defined for U-mode accesses. The `vspmpcfg` registers use the permission encodings `Sspmp` defines for `spmpcfg`, with VS-mode in place of S-mode and VU-mode in place of U-mode.
4. `vsstatus.SUM` takes the role of `sstatus.SUM`.
5. `vsstatus.MXR` takes the role of `sstatus.MXR`.

For accesses with `V=1`, `vsstatus.MXR` affects only the vSPMP stage, whereas the HS-level `sstatus.MXR` affects both the vSPMP and the SPMP stages. In either case, MXR grants read permission to an explicit load only where the permission encoding of that stage grants execute permission to the privilege mode of the load.

The vSPMP is active when `vsatp.MODE` is `Bare`. When `vsatp.MODE` is not `Bare`, VS-stage address translation is in effect instead. Likewise, as defined by `Sspmp`, the SPMP checks accesses whose effective privilege mode is VS or VU only when `hgatp.MODE` is `Bare`. Otherwise, G-stage address

translation is in effect instead. Table 1 lists the resulting logical protection and translation path for every combination. When `V=0`, the vSPMP is never active and the base `Sspmp` rules apply.

Table 1. Logical protection and translation path for accesses with `V=1`

`vsatp.MODE`	`hgatp.MODE`	**Path**
Bare	Bare	vSPMP, then SPMP, then PMP
Bare	not Bare	vSPMP, then G-stage address translation, then PMP
not Bare	Bare	VS-stage address translation, then SPMP, then PMP
not Bare	not Bare	Two-stage address translation, then PMP



Implementations may perform the checks and translations of a row of Table 1 in parallel, provided the overall logical behavior matches the order of that row.

If the vSPMP check, or both the vSPMP and SPMP checks, deny a VS/VU access, a page-fault exception (code 12, 13, or 15) is raised, as defined by `Sspmp`, and taken according to the standard delegation rules. If only the SPMP denies a VS/VU access, a guest-page-fault exception (code 20, 21, or 23) is raised, as defined by `Sspmp` for VS/VU accesses denied by the SPMP, and taken according to the standard delegation rules. Each row of Table 1 lists the stages in order of increasing privilege level, and exceptions are prioritized accordingly: a denial by a stage takes priority over any exception of the stages that follow it in the same row. That is, when more than one stage denies an access, only the exception of the least privileged denying stage is reported.

When the SPMP stage denies a VS/VU access, the resulting guest-page fault follows the standard reporting rules of the Privileged Architecture: `stval` (or `mtval`) is written with the faulting guest virtual address, `htval` (or `mtval2`) with either zero or the faulting guest physical address right-shifted by 2, and `htinst` (or `mtinst`) according to the rules for guest-page faults. When the vSPMP stage denies an access, the resulting page fault writes the faulting guest virtual address to `vstval`, `stval`, or `mtval`, according to the mode in which the trap is taken.

Accesses performed by the hypervisor virtual-machine load and store instructions (HLV, HLVX, HSV), as well as M-mode explicit memory accesses with `mstatus.MPRV=1`, `mstatus.MPV=1`, and `mstatus.MPP` set to S or U, are performed as though `V=1` and thus have an effective privilege mode of VS or VU. They are therefore subject to the vSPMP and SPMP checks.

For HLVX.HU and HLVX.WU, *execute* permission takes the place of *read* permission at the vSPMP and SPMP stages, as it does during address translation, and a denial raises the corresponding load exception.

The vSPMP is configured through a set of virtual supervisor indirect-access registers, which are architectural replicas of the standard SPMP registers, with the same field layout and encodings, and with VSXLEN in place of SXLEN. These include `vspmpaddr0` through `vspmpaddr63` and `vspmpcfg0` through `vspmpcfg63`. See Section 2.3 for details on how these registers are accessed.

2.3. Access Method for vSPMP registers

Consistent with the behavior defined in the `Shbare` and `Sscsrind` extensions, M-mode and HS-mode access the vSPMP registers through the `vsiselect` and `vsireg*` CSRs. VS-mode accesses targeting the SPMP registers through the `siselect` and `sireg*` CSRs are transparently redirected to the corresponding vSPMP registers.

Each `vsiselect` value selects one vSPMP entry, using the same index mapping defined for `siselect` in the base `Sspmp` extension: vSPMP entry `i` ($0 \le i \le 63$) is selected by writing `0x100+i` to `vsiselect`. Indirect accesses via `vsireg` and `vsireg2` access that entry's `vspmpaddr` and `vspmpcfg`, respectively. The registers `vsireg3` through `vsireg6` are reserved.

`vsiselect` **value**	**indirect CSR access of** `vsireg`
`0x100`	`vsireg` → `vspmpaddr[0]`, `vsireg2` → `vspmpcfg[0]`
`0x101`	`vsireg` → `vspmpaddr[1]`, `vsireg2` → `vspmpcfg[1]`
…	…
`0x13F`	`vsireg` → `vspmpaddr[63]`, `vsireg2` → `vspmpcfg[63]`

Both M and HS-mode accesses via `vsireg2` CSRs, and VS-mode accesses via `sireg2` CSRs to the vSPMP `vspmpcfg` registers, can set `vspmpcfg[i].L` to lock a vSPMP entry. VS-mode attempts to write to locked `vspmpcfg[i]` and `vspmpaddr[i]` registers using the `siselect` CSR are ignored. If `vspmpcfg[i].L` is set and `vspmpcfg[i].A` is set to TOR, VS-mode writes to the preceding `vspmpaddr[i-1]` via `siselect` are also ignored. The lock bit `vspmpcfg[i].L` of a vSPMP entry can only be cleared by M or HS-mode through an indirect access using `vsiselect`.

When `hstatus.VTVM` is set and `V=1`, attempts to access `sireg*` (really `vsireg*`) while `siselect` (really `vsiselect`) holds a value in the range `0x100` to `0x13F` raise a virtual-instruction exception. `hstatus.VTVM` must be writable when `Ssvspmp` is implemented, even if `vsatp.MODE` is read-only `Bare`.



Trapping VS-mode accesses to the vSPMP registers allows HS-mode software to emulate the vSPMP, for example to provide a guest with more entries than are delegated to the vSPMP.

Indirect accesses to vSPMP CSRs are not ordered with respect to each other or with subsequent memory accesses. To enforce ordering for subsequent accesses whose effective privilege mode is VS or VU, software must execute memory-management fence instructions. When `V=1`, an `SFENCE.VMA` instruction with `rs1=x0` and `rs2=x0` is required, as defined by `Sspmp`. When `V=0`, M or HS-mode software must execute an `HFENCE.VVMA` instruction with `rs1=x0` and `rs2=x0`, which synchronizes subsequent memory accesses with all preceding vSPMP CSR writes.

Chapter 3. "Ssvspmpen" Extension for Optimizing Context Switching of vSPMP Entries

The `Ssvspmpen` extension is an optional feature designed to enhance the performance of vSPMP context switches performed by VS-mode software, replicating for the vSPMP the functionality that `Sspmpen` provides for the SPMP. This extension is mandatory for implementations that support both `Ssvspmp` and `Sspmpen`. For this, it introduces new virtual supervisor CSRs that control the activation of vSPMP entries:

- For RV64 architectures, it introduces a 64-bit WARL CSR, `vspmpen`.
- For RV32 architectures, it introduces an additional 32-bit WARL CSR, `vspmpenh`, which serves as an alias for the upper 32 bits of the `vspmpen` register.

When `V=1`, `vspmpen` substitutes for the usual `spmpen` (and `vspmpenh` for `spmpenh`), so instructions that normally read or modify `spmpen` actually access `vspmpen` instead.

The rules of `Sspmpen` apply to `vspmpen` and the vSPMP entries, with `vspmpcfg` in place of `spmpcfg`, and with the ordering requirements of Section 2.3 in place of those of the base `Sspmp` extension.

When `hstatus.VTVM` is set and `V=1`, attempts to access `vspmpen` through the `spmpen` (and `spmpenh`) CSRs raise a virtual-instruction exception.



`Ssvspmpen` *does not reduce the cost, for HS-mode software, of switching the vSPMP state when switching between virtual harts, which requires saving and restoring the vSPMP registers of each virtual hart. A mechanism for that purpose may be defined by a future extension.*

Chapter 4. "Sshspmpdeleg" Extension for Sharing Hardware Resources between SPMP and vSPMP

The vSPMP registers are architectural replicas of the SPMP registers, sharing their layout and address-matching logic, which makes hardware reuse between the two a practical approach for resource conservation. This chapter introduces the `Sshspmpdeleg` extension, a mechanism that allows the hypervisor to dynamically allocate the entries delegated to it between the SPMP and the vSPMP. This extension is mandatory for implementations that support `Ssvspmp`.



In practice, the split between the SPMP and the vSPMP is expected to be mostly static: either hardwired by the implementation, or set by software during system initialization and configuration, rather than changed dynamically at runtime. Runtime reconfiguration remains possible, subject to the software coordination described in Section 5.2.

4.1. Resource Sharing between SPMP and vSPMP

The `Sshspmpdeleg` extension introduces a new HSXLEN-bit HS-mode CSR named `hspmpdeleg`, whose layout is detailed in Figure 2. The `hspmpdeleg.pmpnum` WARL field defines the number of SPMP entries reserved for hypervisor use. All SPMP entries with an index greater than or equal to `hspmpdeleg.pmpnum` are delegated to vSPMP.

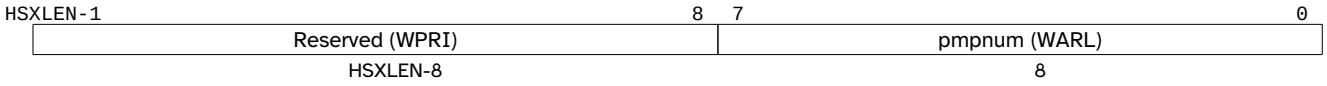


Figure 2. hspmpdeleg CSR format.

This extension allows for the total number of implemented entries across PMP, SPMP, and vSPMP to be greater than 64 and up to a maximum of 192.



The limit of 192 stems from the maximum number of addressable entries across all three levels: 64 for PMP, 64 for SPMP, and 64 for vSPMP.

If a write to `hspmpdeleg.pmpnum` specifies a value exceeding the number of SPMP entries provided by the HEE, the field subsequently reads back that total count. In such a case, no entries are delegated to vSPMP.

If no entries are delegated to the vSPMP, the `Ssvspmp` extension is effectively disabled.



The vSPMP can be effectively disabled by setting `hspmpdeleg.pmpnum` to the total number of SPMP entries provided by the HEE.



The `hspmpdeleg.pmpnum` field is a WARL field, which allows an implementation to hardwire the partition between SPMP and vSPMP.

vSPMP entries are numbered contiguously, with `vSPMP[0]` selected by the lowest `vsiselect` value, `0x100`. `SPMP[hspmpdeleg.pmpnum]` through `SPMP[N-1]` map to `vSPMP[0]` through `vSPMP[N-1-hspmpdeleg.pmpnum]`, where N is the total number of SPMP entries provided by the HEE. For instance, given 32 SPMP entries provided by the HEE and `hspmpdeleg.pmpnum` set to 16, `SPMP[16]` to `SPMP[31]` map to `vSPMP[0]` to `vSPMP[15]`. A read of a vSPMP entry with an index beyond the delegated range returns `0`, and a write to such an entry is ignored. Bits of `spmpen`, `hspmpen`, and `vspmpen` beyond the current SPMP and vSPMP ranges read as zero, and writes to them are ignored.

Only the lowest 64 delegated entries are addressable through the vSPMP CSRs. Any additional delegated entries are not addressable and are therefore inert, as defined in Section 5.1. Similarly, if more than 64 entries are reserved for SPMP (i.e., `hspmpdeleg.pmpnum` > 64), only the lowest 64 are addressable through the SPMP CSRs, and the remaining reserved entries are inert. These limits apply to each view separately, so no entry is lost as long as no view is assigned more than 64 entries. The split among the PMP, the SPMP, and the vSPMP is described in Section 5.2.

Writes to `hspmpdeleg` are ordered with respect to subsequent indirect accesses to SPMP and vSPMP entries, which use the new mapping, and to subsequent accesses to `spmpen`, `hspmpen`, and `vspmpen`. They are not ordered with respect to subsequent memory accesses. Before resuming execution that depends on the new allocation, software must execute the fence instructions required for SPMP CSR writes, as defined by `Sspmp`, and for vSPMP CSR writes, as defined in Section 2.3.

The `hspmpdeleg.pmpnum` value cannot be set to an index that is less than or equal to that of any locked addressable SPMP entry. For example, if `SPMP[7]` is locked, any attempt to write a value less than 8 to `hspmpdeleg.pmpnum` is ignored, and the field retains its prior value.

The `hspmpdeleg.pmpnum` value can be set to override locked vSPMP entries. For example, if `vSPMP[0]` is locked, HS-mode software can still increment `hspmpdeleg.pmpnum`.



Moving a locked SPMP entry into the vSPMP would remove its rule from the SPMP stage and would allow HS-mode to clear its lock through `vsiselect`*, defeating the lock. Only a more privileged level, M-mode through* `mpmpdeleg`*, may change the role of a locked SPMP entry.*

Chapter 5. "Smhspmpdeleg" Extension for M-mode Support of vSPMP Resource Sharing

With Sshspmpdeleg, the entries delegated by M-mode are further divided between the SPMP and the vSPMP, so that a single pool of hardware entries serves three protection levels. This chapter introduces the Smhspmpdeleg extension, which extends mpmpdeleg to address pools larger than the 64 entries of the base Smpmpdeleg extension and defines how the M-mode and HS-mode allocations interact. This extension is mandatory for implementations that support Sshspmpdeleg in conjunction with M-mode (i.e., Sm1p13).

5.1. Extension to mpmpdeleg

When Smhspmpdeleg is implemented, mpmpdeleg gains an ext enable bit at bit 8 and extends pmpnum to 8 bits. Bits 7 and 8 were previously reserved (WPRI) in Smpmpdeleg. The extended layout is shown in Figure 3.

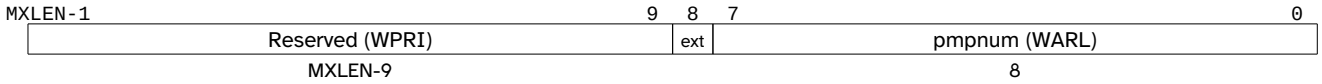


Figure 3. Extended mpmpdeleg CSR format.

The implemented entries are the entries provided by the hardware, numbered contiguously from 0 to N-1, where N is at most 192. If N is greater than 64, entries 0 to 63 are all implemented. The writable PMP entries, as defined by the base Smpmpdeleg extension, are the implemented entries that can be delegated: the implemented entries with index below 64 when ext is clear, and all implemented entries when ext is set.

An entry is addressable when it can be accessed through the PMP, SPMP, or vSPMP CSRs. Which entries are addressable depends on ext and on the current values of mpmpdeleg.pmpnum and hspmpdeleg.pmpnum. Implemented entries that are not writable, and writable entries that are not addressable, are inert: they perform no checks, have no effect on memory protection, and retain their register contents.

When ext is clear (the reset state), the base Smpmpdeleg rules apply unchanged: bit 7 of pmpnum is read-only zero, and pmpnum is capped at the number of writable PMP entries, which is at most 64. Implemented entries with index 64 and above are not writable, are not delegated to any level, and are therefore inert. When ext is set, bit 7 of pmpnum becomes writable and pmpnum is capped at the number of writable PMP entries, which is then N, so that any implemented entry can be delegated, subject to the limit of 64 addressable entries per view described in Section 4.1. In both cases, a write to pmpnum of a value exceeding its cap causes the field to read back the cap.

Once set, ext cannot be cleared until reset. A write that sets ext takes effect together with the pmpnum value written by the same instruction, and that value is validated against the new ext setting. Setting ext makes the implemented entries with index 64 and above writable. They are assigned roles according to mpmpdeleg.pmpnum and hspmpdeleg.pmpnum as described in Section 5.2.

The ext bit is writable regardless of the number of implemented entries. When 64 or fewer entries are implemented, setting ext has no effect.



The base `Smpmpdeleg` *defines* `pmpnum` *as 7 bits, sufficient to represent values up to 127 and cover the 64-entry base case.* `Smhspmpdeleg` *raises the total entry count to up to 192, which cannot be represented in 7 bits. Extending* `pmpnum` *to 8 bits (max 255) allows M-mode to set* `mpmpdeleg.pmpnum` *equal to the total number of implemented entries, thereby delegating nothing to SPMP and effectively disabling both SPMP and vSPMP. Without this extension, M-mode would be unable to reclaim all entries as PMP when more than 64 entries are implemented.*

Because `ext` *resets to* `0` *and software compliant with* `Smpmpdeleg` *preserves WPRI bits on write, existing M-mode software unaware of* `Smhspmpdeleg` *will always observe* `pmpnum` *behave as a 7-bit field. Setting* `ext` *requires explicit action by* `Smhspmpdeleg` *-aware software.*

5.2. M-mode Support for Hypervisor/Guest SPMP Resource Sharing

When both `Smpmpdeleg` and `Sshspmpdeleg` are implemented, the available hardware entries are divided three ways between PMP, SPMP, and vSPMP. The `mpmpdeleg.pmpnum` field defines the PMP/SPMP boundary, and `hspmpdeleg.pmpnum` further divides the SPMP entries between hypervisor and guest use. Any entry with an index greater than or equal to `mpmpdeleg.pmpnum` + `hspmpdeleg.pmpnum` is delegated to the vSPMP. For example, if the number of implemented entries is 48, `mpmpdeleg.pmpnum` is set to 8, and `hspmpdeleg.pmpnum` is set to 16, entries 24 to 47 are delegated to the vSPMP.

The M-mode allocation always takes precedence over the HS-mode allocation. If a write to `hspmpdeleg.pmpnum` makes `mpmpdeleg.pmpnum` + `hspmpdeleg.pmpnum` greater than the number of writable PMP entries, the field reads back as the number of remaining entries. A write to `mpmpdeleg` updates `hspmpdeleg.pmpnum` so that the sum `mpmpdeleg.pmpnum` + `hspmpdeleg.pmpnum` is unchanged, which preserves the entries already delegated to the vSPMP and their indices. If the new `mpmpdeleg.pmpnum` is greater than the previous sum, `hspmpdeleg.pmpnum` keeps its previous value instead. In either case, `hspmpdeleg.pmpnum` is capped at the number of remaining entries. For example, if the number of implemented entries is 32, `mpmpdeleg.pmpnum` is set to 16, and then `hspmpdeleg.pmpnum` is written a value of 32, the field will read back as 16. If `mpmpdeleg.pmpnum` is set to 8 and `hspmpdeleg.pmpnum` to 20, and then `mpmpdeleg.pmpnum` is written 16, `hspmpdeleg.pmpnum` is updated to 12, and entries 28 to 31 remain delegated to the vSPMP. If `mpmpdeleg.pmpnum` is then written 30, `hspmpdeleg.pmpnum` is updated to 2, and no entries remain delegated to the vSPMP. From the reset state, a write of `0` to `mpmpdeleg.pmpnum` updates `hspmpdeleg.pmpnum` to 32, delegating all entries to the SPMP.

Unless hardwired, `mpmpdeleg.pmpnum` resets to the total number of writable PMP entries, so that no entries are delegated to the SPMP at reset, and `hspmpdeleg.pmpnum` resets to `0`.

`hspmpdeleg.pmpnum` may only be hardwired if `mpmpdeleg.pmpnum` is also hardwired.

With the three-way split in place, entries are assigned contiguously to the PMP starting from the lowest index, followed by the SPMP, then the vSPMP. For example, given 48 implemented entries, if `mpmpdeleg.pmpnum` is set to 16 and `hspmpdeleg.pmpnum` to 16, entries 32 to 47 map to `vSPMP[0]` to `vSPMP[15]`.

If more than 64 entries are reserved for PMP (i.e., `mpmpdeleg.pmpnum` > 64), only the lowest 64 are addressable through the PMP CSRs, and the remaining reserved entries are inert. The corresponding rule for the SPMP and vSPMP is given in Section 4.1. For example, if the number of implemented entries is 128, `mpmpdeleg.pmpnum` is set to 16, and `hspmpdeleg.pmpnum` is set to 16, entries 32 to 95 map to `vSPMP[0]` to `vSPMP[63]`, and entries 96 to 127 are not addressable and are therefore inert. Setting

`hspmpdeleg.pmpnum` to 48 instead yields 16 PMP, 48 SPMP, and 64 vSPMP entries, all of them addressable.

Reconfiguration of delegated entries:

At any given time, each implemented entry serves exactly one of three roles, determined by the two delegation fields: entries with index below `mpmpdeleg.pmpnum` serve as PMP, entries from `mpmpdeleg.pmpnum` up to but excluding `mpmpdeleg.pmpnum` + `hspmpdeleg.pmpnum` serve as SPMP, and the remaining entries serve as vSPMP. Writing either delegation field changes the role of the entries that cross a boundary.

1. `PMP[i]` is entry i, `SPMP[k]` is entry `mpmpdeleg.pmpnum` + k, and `vSPMP[j]` is entry `mpmpdeleg.pmpnum` + `hspmpdeleg.pmpnum` + j. As either field is reconfigured, the entries crossing a boundary change role, and the entries above it are reached at new indices within their roles, consistent with the reconfiguration rules of the base `Smpmpdeleg` extension. A change to `mpmpdeleg.pmpnum` alone keeps the vSPMP indices, unless the new value is greater than the previous `mpmpdeleg.pmpnum` + `hspmpdeleg.pmpnum`.
2. Raising `hspmpdeleg.pmpnum` from N to N+2 expands the SPMP range from `SPMP[0..N-1]` to `SPMP[0..N+1]`, the former `vSPMP[0]` and `vSPMP[1]` becoming `SPMP[N]` and `SPMP[N+1]`, and truncates the vSPMP range from `vSPMP[0..M-1-N]` to `vSPMP[0..M-3-N]`, where M is the number of entries delegated by `mpmpdeleg`. Reducing `hspmpdeleg.pmpnum` has the converse effect.
3. The `spmpen`, `hspmpen`, and `vspmpen` ranges truncate and expand together with their corresponding SPMP and vSPMP entry ranges.
4. Entries that change role retain their address register, configuration register, and enable bits, as their storage is shared across roles. An entry's `spmpen` and `vspmpen` bits share the same storage, and its `hspmpen` bit has separate storage.

`hspmpdeleg.pmpnum` is allowed to be updated to a value less than or equal to the index of any locked SPMP entry only due to a write to `mpmpdeleg.pmpnum`.

Writes to `mpmpdeleg`, including `ext`, are ordered with respect to subsequent accesses to PMP, SPMP, and vSPMP entries, which use the new mapping, and any resulting adjustment of `hspmpdeleg.pmpnum` takes effect together with the write. Such a write is also an implicit write to `hspmpdeleg`, and the ordering rules for writes to `hspmpdeleg` (see Section 4.1) apply to it as well. Writes to `mpmpdeleg` are not ordered with respect to subsequent memory accesses. Before resuming execution that depends on the new allocation, software must execute the fence instructions required for PMP CSR writes, as defined by the Privileged Architecture, for SPMP CSR writes, as defined by `Sspmp`, and for vSPMP CSR writes, as defined in Section 2.3.



Because entries keep their contents across role changes, an entry programmed by one level would otherwise be enforced as a rule of another level as soon as the boundary moves. Software changing `mpmpdeleg` *(including* `ext`*) or* `hspmpdeleg` *should quiesce the affected execution and memory accesses and perform the reconfiguration as a non-preemptible sequence, preserving the permissions of the reconfiguration code. Before the reallocation, software should disable the addressable affected entries by setting their* `A` *field to OFF, if not hardwired, or, when an enable mask is implemented, by clearing their enable bit, and clear their* `L` *bit where permitted. Entries that move to a higher privilege level are addressable by the level performing the reconfiguration beforehand, through the indirect access registers of the level they leave, so they can always be disabled at this point. After the reallocation, the responsible privilege level should configure the newly addressable*

or reassigned entries, correct the TOR ranges affected by renumbering, and complete the required synchronization before resuming the affected execution. Entries that become addressable only after the reallocation always belong to a lower privilege level than the one performing the reconfiguration, do not affect its execution, and are configured by the receiving level at this point.

A write to `mpmpdeleg.pmpnum` *can change the value of* `hspmpdeleg.pmpnum` *and the SPMP/vSPMP index mappings without any notification to HS-mode. It is the responsibility of M-mode software to coordinate such reconfigurations with the hypervisor, through a protocol outside the scope of this specification (for example, an SBI extension), and of HS-mode software to re-read* `hspmpdeleg` *after any M-mode reconfiguration it is aware of. Through this protocol, HS-mode relinquishes the affected entries and quiesces the affected guests before the reconfiguration takes place.*

Entries that M-mode releases by lowering `mpmpdeleg.pmpnum` *are delegated to the SPMP. Since no entries are delegated to the vSPMP at reset, M-mode software unaware of* `Sshspmpdeleg` *thus delegates entries to HS-mode as with* `Smpmpdeleg` *alone. Setting* `ext` *without changing* `mpmpdeleg.pmpnum` *delegates the newly writable entries to the vSPMP. M-mode software that reserves them for the hypervisor is expected to write* `hspmpdeleg.pmpnum` *after* `mpmpdeleg`*, within the same non-preemptible reconfiguration sequence and before resuming affected execution.*

Chapter 6. "Sshspmpen" Extension for Optimizing Context Switching of SPMP Hypervisor/Guest Entries

The `Sshspmpen` extension is an optional feature that provides separate enable masks for the SPMP entries applied to guest accesses. It depends on the `Sspmpen` and `Shbare` extensions. It introduces additional CSRs:

- For RV64 architectures, it introduces a 64-bit WARL CSR, `hspmpen`.
- For RV32 architectures, it introduces an additional 32-bit WARL CSR, `hspmpenh`, which serves as an alias for the upper 32 bits of the `hspmpen` register.

The `hspmpen` register takes effect only when enabled through the `HSPMPEN` bit of the `henvcfg` register (bit 58, provisional pending allocation). When `henvcfg.HSPMPEN` is clear (the reset state), `hspmpen` has no effect and `spmpen` governs the active SPMP entries regardless of V, as when `Sshspmpen` is not implemented. When `henvcfg.HSPMPEN` is set and `V=1`, `hspmpen` replaces the function of `spmpen`: an SPMP entry is considered active only if both its `spmpcfg[i].A` field is non-zero and the corresponding bit in `hspmpen` is set. When `V=0`, the `hspmpen` register has no effect, and only `spmpen` governs the active SPMP entries, as defined by `Sspmpen`. The `hspmpen` register is accessible regardless of `henvcfg.HSPMPEN`.

When an entry `i` is locked, as indicated by `spmpcfg[i].L == 1`, bit `i` in `hspmpen` becomes read-only. When any SPMP entry is locked, any further modifications to `henvcfg.HSPMPEN` are ignored, so that locking an entry also fixes which enable mask governs it for accesses with `V=1`.

The `hspmpen[i]` bit controls only whether SPMP entry `i` participates in address matching and enforcement for accesses with `V=1`. It does not control whether `spmpaddr[i]` or `spmpcfg[i]` is writable. Writes to the `hspmpen` CSR and to `henvcfg.HSPMPEN` are not ordered with respect to subsequent memory accesses. To enforce ordering for subsequent accesses whose effective privilege mode is VS or VU, software must execute an `HFENCE.GVMA` instruction with `rs1=x0` and `rs2=x0`, as defined by `Sspmp`.



Because `HSPMPEN` *resets to 0 and software unaware of* `Sshspmpen` *preserves reserved* `henvcfg` *bits on write, the presence of this extension does not change the behavior observed by hypervisors aware only of* `Sspmp` *and not of this extension.* `hspmpen` *takes effect only after explicit opt-in by* `Sshspmpen`*-aware software.*



The rationale for providing a separate `hspmpen` *register, rather than reusing* `spmpen`*, is twofold.*

First, HS/U and VS/VU contexts conceptually occupy distinct address spaces, not merely different permission domains. This mirrors the Hypervisor extension for page-based translation, where HS/U accesses use the page tables pointed to by `satp`*, while VS/VU accesses are translated through the separate VS-stage and G-stage page tables defined by* `vsatp` *and* `hgatp`*. Similarly,* `spmpen` *governs the SPMP entries protecting the hypervisor's own address space, while* `hspmpen` *controls those enforced on the guest address space, both selecting subsets of the same SPMP entries.*

Second, hardware selects between the two registers based on V, which makes transitions between virtualization modes both safe and free of CSR traffic. Upon a trap into HS-mode with `satp.MODE` *set to* `Bare`*, the first instructions of the trap handler are already subject to SPMP checks with* `V=0`*. If the enable mask were shared, those instructions would be checked against the mask configured for the guest, forcing the hypervisor to keep a single mask valid for both virtualization modes*

and constraining how entries can be partitioned.